
ARTIE SHEVCHENKO

PROGRAMADORES COMO GUARDIANES DE LA SALUD DEL CÓDIGO

Claudia Rodríguez Fuertes - UO288406

Mario Amandi San Marín - UO302457

Sergio García Santamarina - UO294636

```
mirror_mod = modifier_ob.  
Get mirror object to mirror  
mirror_mod.mirror_object =  
operation == "MIRROR_X":  
mirror_mod.use_x = True  
mirror_mod.use_y = False  
mirror_mod.use_z = False  
operation == "MIRROR_Y":  
mirror_mod.use_x = False  
mirror_mod.use_y = True  
mirror_mod.use_z = False  
operation == "MIRROR_Z":  
mirror_mod.use_x = False  
mirror_mod.use_y = False  
mirror_mod.use_z = True
```

```
selection at the end -add  
mirror_ob.select= 1  
modifier_ob.select=1  
context.scene.objects.active  
("Selected" + str(modifier_ob.  
mirror_ob.select = 0  
= bpy.context.selected_object  
data.objects[one.name].select  
print("please select exactly
```

-- OPERATOR CLASSES --

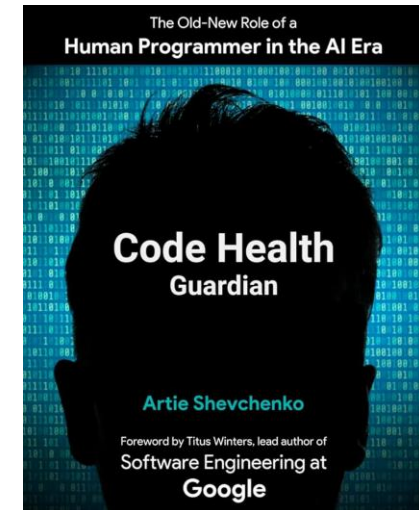
```
types.Operator):  
on X mirror to the selected  
object.mirror_mirror_x"  
mirror X"
```

```
context):  
context.active_object is not
```

ARTIE SHEVCHENKO

- Ex ingeniero de software de Google
- Ex CEO y fundador de un startup
- Actualmente, ingeniero de software en Canva y profesor en la Universidad ITMO

Autor del libro recientemente publicado Code Health Guardian, the Old New Role of Human Programmers in the AI Era



¿QUÉ SE NECESITA PARA QUE LOS PROGRAMADORES TENGAN ÉXITO EN LA ERA DE LA IA?

- Colaborar con la IA.
- Delegar en la IA.
- Gestión de la complejidad.

Code Health Guardian

¿POR QUÉ LE CUESTA A LA IA MANTENER LAS COSAS SIMPLES?



- No entiende completamente el contexto histórico.



- No anticipa efectos secundarios profundos.



- No gestiona bien la evolución del sistema en el tiempo.

CODE HEALTH

- Equilibrio entre productividad a corto plazo y productividad a largo plazo.



LOS 3 PROBLEMAS DE LA COMPLEJIDAD

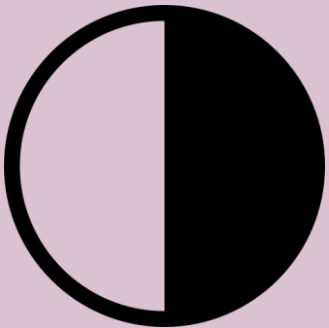
Change Amplification
(Amplificación de cambios)

Cognitive Load (Carga
cognitiva)

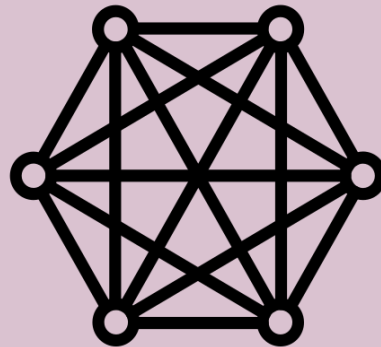
Unknown Unknowns (Lo
que no sabes que no sabes)

CAUSAS PRINCIPALES DE LA COMPLEJIDAD

OSCURIDAD



INTERFACES
COMPLEJAS



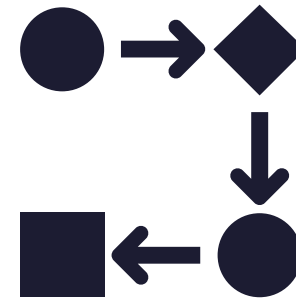
FALTA DE
FAMILIARIDAD



PRINCIPIOS DE DISEÑO A USAR EN ESTA NUEVA FUNCIÓN DEL PROGRAMADOR COMO CODE HEALTH GUARDIAN



- Diseño por Contrato



- DRY(Don't Repeat Yourself)

DISEÑO POR CONTRATO

Contratos claros que cumpla la interfaz

Uso de precondiciones, postcondiciones e invariantes

No sirve con hacer solo test

- Es mejor no tragarse excepciones y fallar explícitamente y cuanto antes mejor

DON'T REPEAT YOURSELF DESDE LA PERSPECTIVA DE LA COMPLEJIDAD



Duplicación de Código



Carga cognitiva



Rango del cambio (change amplification)



Unkown unknown (EL PEOR)



Consejos

Eliminar los unknkown unknown.
Evaluar el coste-beneficio del
cambio.
Eliminar duplicación que lleve a
simplificación.

PROCESOS PARA MANTENER CÓDIGO SANO

Design Docs

- BIG UPFRONT DESIGN

Revisión de código

- CALIDAD DEL CÓDIGO
 - COMPARTIR CONOCIMIENTO
 - VELOCIDAD DE ENTREGA
 - SEGURIDAD PSICOLÓGICA
-